

Vehicle ECU Simulator

Technical Documentation

ISO C99 Embedded Systems Project

Language	C (ISO C99)
Standard	MISRA-C / CERT-C Compliant
Compiler	GCC with <code>-Wall</code> <code>-Wextra</code> <code>-pedantic</code>
Platform	Linux / macOS / WSL (Windows)
Version	1.0
Date	April 16, 2026

*A modular, cyclic Electronic Control Unit (ECU) simulator
that enforces safety thresholds, processes sensor inputs,
manages vehicle modes, and evaluates system health
through fault persistence tracking.*

Repository: `chack_ecu`

Contents

1	Project Overview	5
1.1	Introduction	5
1.2	System Goals	5
1.3	Repository Structure	6
2	Architecture	7
2.1	Cyclic Execution Pipeline	7
2.2	Architecture Diagram	9
2.3	Module Dependency Graph	9
3	Data Types and Shared Definitions (<code>types.h</code>)	11
3.1	Overview	11
3.2	Enumerations	11
3.2.1	<code>Mode</code> — Vehicle Operating Mode	11
3.2.2	<code>SystemState</code> — Overall Vehicle Health	12
3.2.3	<code>FaultPriority</code> — Reporting Priority	12
3.3	Structures	12
3.3.1	<code>VehicleInput</code> — Sensor Snapshot	12
3.3.2	<code>FaultStatus</code> — Fault Tracking State	12
3.3.3	<code>VehicleStatus</code> — Vehicle Status	13
3.4	Fault Bitmask Definitions	13
4	Module Documentation	14
4.1	Input Module (<code>input.h</code> / <code>input.c</code>)	14
4.1.1	Responsibility	14
4.1.2	Configuration Constants	14
4.1.3	Public API	15
4.1.4	Validation Logic	15
4.2	Mode Module (<code>mode.h</code> / <code>mode.c</code>)	15
4.2.1	Responsibility	15
4.2.2	State Machine	16
4.2.3	Legal Transition Matrix	16
4.2.4	Public API	17
4.3	Control Module (<code>control.h</code> / <code>control.c</code>)	17
4.3.1	Responsibility	17
4.3.2	Safety Thresholds	17

4.3.3	Fault Priority Logic	18
4.3.4	Public API	18
4.4	Fault Module (<code>fault.h</code> / <code>fault.c</code>)	18
4.4.1	Responsibility	18
4.4.2	Persistence Mechanism	18
4.4.3	Configuration Constants	19
4.4.4	Public API	19
4.5	State Module (<code>state.h</code> / <code>state.c</code>)	20
4.5.1	Responsibility	20
4.5.2	State Machine	20
4.5.3	State Transition Rules	21
4.5.4	Configuration Constants	21
4.5.5	Public API	22
4.6	Log Module (<code>log.h</code> / <code>log.c</code>)	22
4.6.1	Responsibility	22
4.6.2	Cycle Summary Sections	22
4.6.3	Fault Flag Display Format	23
4.6.4	Public API	23
5	Main Execution Pipeline (<code>main.c</code>)	24
5.1	Entry Point	24
5.2	Watchdog Logic	24
5.3	Local Variables	25
6	Build and Usage	26
6.1	Requirements	26
6.2	Compilation	26
6.3	Running the Simulator	26
6.4	Input Format	27
6.5	Walkthrough: Normal Vehicle Start-Up	27
6.6	Walkthrough: Fault Scenarios	28
6.7	Watchdog Reset Procedure	28
7	Safety and Coding Standards	29
7.1	MISRA-C Compliance Practices	29
7.2	CERT-C Compliance Practices	29
7.3	Defensive Design Patterns	30
8	Output Log Files	31
8.1	<code>valid_input.txt</code>	31
8.2	<code>faults_errors.txt</code>	31
9	Glossary	32

List of Tables

2.1	Cyclic Execution Pipeline — Step Overview	8
3.1	<code>Mode</code> Enumeration Values	11
3.2	<code>SystemState</code> Enumeration Values	12
3.3	<code>FaultPriority</code> Enumeration Values	12
3.4	Fault Bitmask Macro Definitions	13
4.1	Input Module Constants (<code>input.h</code>)	14
4.2	Input Module Public Functions	15
4.3	Mode Transition Allowed Matrix (1 = allowed, 0 = illegal → FAULT) . . .	16
4.4	Mode Module Public Functions	17
4.5	Control Module Safety Thresholds (<code>control.h</code>)	17
4.6	Control Module Public Function	18
4.7	Fault Module Constants (<code>fault.h</code>)	19
4.8	Fault Module Public Functions	19
4.9	State Transition Conditions	21
4.10	State Module Thresholds (<code>state.h</code>)	21
4.11	State Module Public Functions	22
4.12	Log Module Public Function	23
5.1	<code>main.c</code> Local Variables	25
6.1	Compiler Flags	26
6.2	Input Field Specification	27
6.3	Example: Normal Start-Up Sequence	27
6.4	Key Fault Scenarios	28
7.1	Defensive Programming Patterns Used	30
9.1	Glossary of Terms	32

List of Figures

2.1	Cyclic ECU Pipeline — execution flow per cycle	9
2.2	Module dependency graph — arrows denote “uses”	10
4.1	Mode state machine — legal transitions and forced FAULT paths	16
4.2	System state machine — transition conditions	20

Chapter 1

Project Overview

1.1 Introduction

The **Vehicle ECU Simulator** is a modular, cyclic Electronic Control Unit (ECU) simulator written in ISO C99. It replicates the core processing pipeline of a real automotive ECU, enforcing safety thresholds, managing vehicle operating modes, processing sensor inputs, and tracking fault persistence over time.

The simulator is built to a strict modular architecture that mirrors industrial embedded systems standards. The codebase is designed with safety-critical practices in mind, following MISRA-C and CERT-C guidelines (no magic numbers, NULL-pointer guards on every public function, explicit type width via `<stdint.h>`).

1.2 System Goals

- Simulate the cyclic execution pipeline of a vehicle ECU in real time.
- Validate sensor inputs (speed, temperature, gear, mode) against hardware bounds.
- Enforce a legal state machine for vehicle operating modes.
- Detect and classify faults based on physical thresholds.
- Track fault persistence — a warning must repeat across three consecutive cycles before being “latched” as a persistent fault.
- Escalate overall system health: **NORMAL** → **DEGRADED** → **SAFE**.
- Produce a structured diagnostic log every cycle.
- Support a watchdog-triggered system reset via a safe input sentinel or consecutive **SAFE** cycles.

1.3 Repository Structure

```
chack_ecu/
include/      # Public module header files
  types.h     # Centralised structs, enums, and bitmask definitions
  input.h     # Input reading and validation API
  mode.h      # Mode state machine API
  control.h   # Control / threshold checking API
  fault.h     # Fault tracking and persistence API
  state.h     # System state escalation API
  log.h       # Cycle summary logging API
src/          # C implementation files
  main.c      # Cyclic execution pipeline (entry point)
  input.c     # Input reading and validation logic
  mode.c      # Mode state machine implementation
  control.c   # Control checks against safety thresholds
  fault.c     # Fault bit management and persistence logic
  state.c     # System state machine and initialisation
  log.c       # Formatted cycle summary output
valid_input.txt # Timestamped log of all validated sensor inputs
faults_errors.txt # Timestamped log of all fault events
README.md     # Quick-start guide
```

Chapter 2

Architecture

2.1 Cyclic Execution Pipeline

The ECU simulator executes a strict, sequential pipeline on every cycle. The pipeline is orchestrated by the `main()` function in `src/main.c` and consists of seven ordered steps:

Table 2.1: Cyclic Execution Pipeline — Step Overview

Step	Function	Description
1	<code>read_inputs()</code>	Reads raw sensor data from <code>stdin</code> : Speed, Temperature, Gear, Mode.
W	Watchdog Check	Detects the all-zero sentinel (0 0 0 0) or two consecutive SAFE cycles; if either condition is met, triggers <code>init_system()</code> to reset the ECU.
2	<code>validate_inputs()</code>	Validates each field against hardware bounds. Out-of-range values are corrected to the last valid reading; invalid gear/mode bits are flagged immediately.
3	<code>update_mode()</code>	Checks the requested mode against the legal transition table. An illegal transition forces the active mode to FAULT and increments the major fault counter.
4	<code>run_control_checks()</code>	Compares speed and temperature against safety thresholds and sets the appropriate fault bits and priority level.
5	<code>update_fault_status()</code>	Iterates through all tracked fault bits; increments per-fault counters, and latches a fault into <code>persistent_flags</code> when its counter reaches the persistence threshold (3 cycles).
6	<code>evaluate_system_state()</code>	Drives the overall system state machine (NORMAL / DEGRADED / SAFE) based on accumulated fault counts.
7	<code>log_cycle_summary()</code>	Prints a full diagnostic summary to <code>stdout</code> and appends structured records to <code>valid_input.txt</code> and <code>faults_errors.txt</code> .

2.2 Architecture Diagram

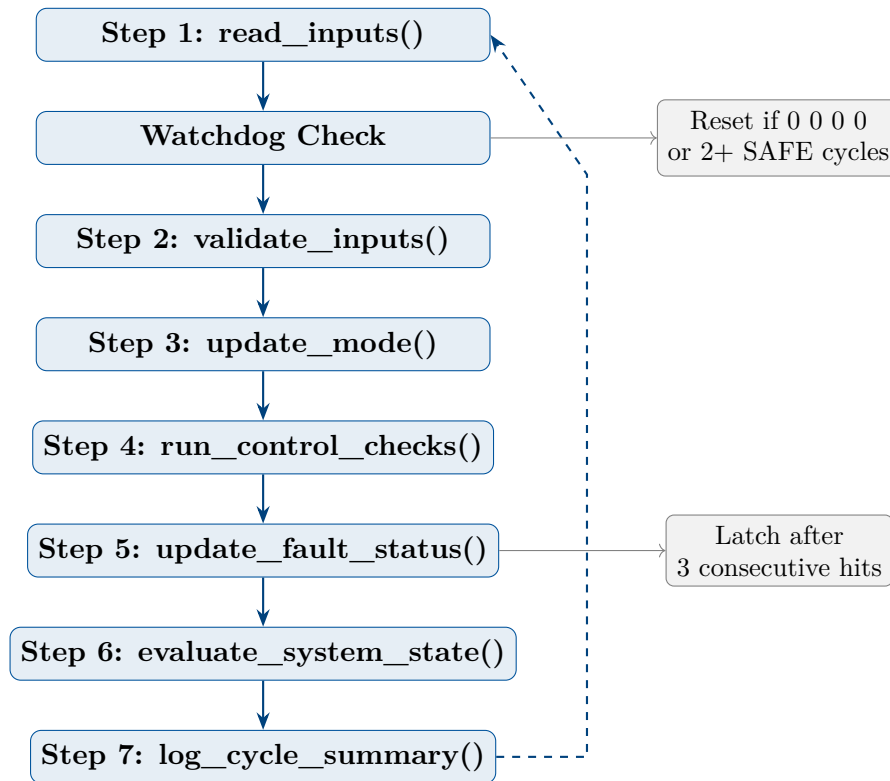


Figure 2.1: Cyclic ECU Pipeline — execution flow per cycle

2.3 Module Dependency Graph

The module architecture forms a clean directed acyclic graph (DAG) with no circular dependencies. `types.h` is the sole shared data contract used by every other module.

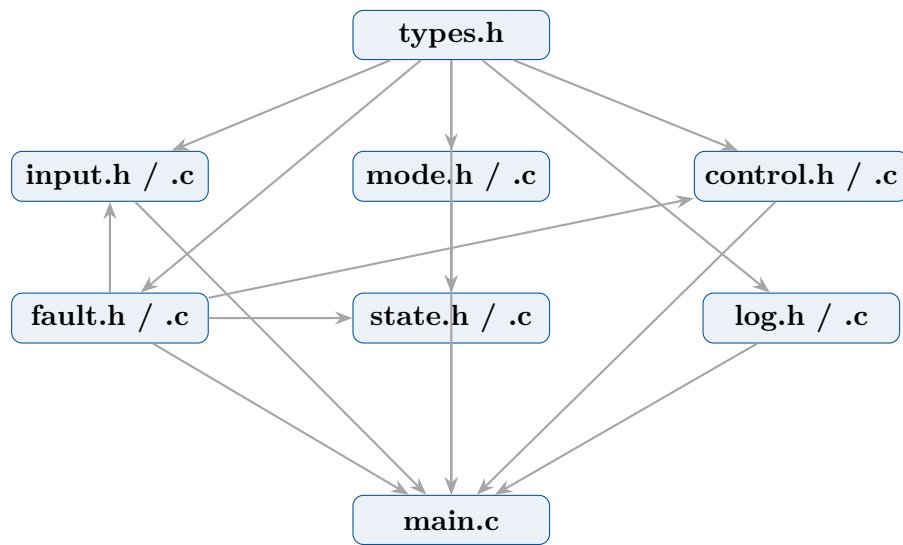


Figure 2.2: Module dependency graph — arrows denote “uses”

Chapter 3

Data Types and Shared Definitions (types.h)

3.1 Overview

`include/types.h` is the single shared data contract for the entire system. It defines all enumerations, structures, and fault bitmask macros. No module communicates through global variables; all state is passed by pointer to these centralised types.

3.2 Enumerations

3.2.1 Mode — Vehicle Operating Mode

Table 3.1: Mode Enumeration Values

Value	Identifier	Meaning
0	MODE_OFF	Engine off; no power to drive systems.
1	MODE_ACC	Accessory power on; electronics active.
2	MODE_IGNITION_ON	Engine running; full operational mode.
3	MODE_FAULT	System forced into fault due to illegal transition or out-of-range mode value.
4	MODE_INVALID	Sentinel; any raw input $\geq$ this value is rejected.

3.2.2 SystemState — Overall Vehicle Health

Table 3.2: SystemState Enumeration Values

Value	Identifier	Meaning
0	NORMAL	No active faults; vehicle operating normally.
1	DEGRADED	One or more major faults or repeated warnings detected; reduce performance.
2	SAFE	Critical fault threshold reached; system must be brought to a safe halt and reset.

3.2.3 FaultPriority — Reporting Priority

Table 3.3: FaultPriority Enumeration Values

Value	Identifier
0	PRIORITY_NONE
1	PRIORITY_HIGH_TEMP
2	PRIORITY_OVERSPEED
3	PRIORITY_INVALID_GEAR_MODE
4	PRIORITY_CRITICAL_OVERHEAT

3.3 Structures

3.3.1 VehicleInput — Sensor Snapshot

```

1 typedef struct {
2     uint16_t speed;           /* Vehicle speed in km/h */
3     int16_t  temperature;    /* Engine temperature in C */
4     uint8_t  gear;           /* Current gear (0 = neutral) */
5     Mode     mode;           /* Requested operating mode */
6 } VehicleInput;

```

Listing 3.1: VehicleInput struct definition

3.3.2 FaultStatus — Fault Tracking State

```

1 typedef struct {
2     uint8_t  major_fault_count; /* Cumulative major faults this
   cycle */
3     uint8_t  warning_count;     /* Cumulative warnings this cycle
   */
4     uint8_t  critical_fault_count; /* Cumulative critical faults
   */

```

```

5     uint8_t  reset_requested;      /* 1 if a reset has been triggered
   */
6     uint16_t current_cycle_flags;  /* Bitmask: faults active this cycle
   */
7     uint16_t persistent_flags;    /* Bitmask: latched persistent
   faults */
8     uint8_t  counters[16];        /* Per-fault consecutive cycle
   counters */
9 } FaultStatus;

```

Listing 3.2: FaultStatus struct definition

3.3.3 VehicleStatus — Vehicle Status

```

1 typedef struct {
2     SystemState  system_state;
3     Mode         active_mode;
4     Mode         current_mode;
5     Mode         previous_mode;
6     FaultPriority highest_priority_issue;
7 } VehicleStatus;

```

Listing 3.3: VehicleStatus struct definition

3.4 Fault Bitmask Definitions

Table 3.4: Fault Bitmask Macro Definitions

Macro	Bit	Description
FAULT_BIT_OVERSPEED	Bit 0	Speed exceeds 120 km/h
FAULT_BIT_CRITICAL_OVERHEAT	Bit 1	Temperature exceeds 110°C
FAULT_BIT_HIGH_TEMP	Bit 2	Temperature in range (95, 110]°C
FAULT_BIT_INVALID_GEAR	Bit 3	Gear value > 5
FAULT_BIT_INVALID_MODE	Bit 4	Mode value ≥ MODE_INVALID
FAULT_BIT_WARNING_TEMP	Bit 5	Temperature in range [100, 110)°C (warning)
FAULT_BIT_WARNING_SPEED	Bit 6	Speed in range [110, 120] km/h (warning)

Chapter 4

Module Documentation

4.1 Input Module (`input.h` / `input.c`)

4.1.1 Responsibility

The input module is the first stage of the pipeline. It is responsible for two distinct operations:

1. **Reading** raw sensor data from `stdin` and populating a `VehicleInput` struct.
2. **Validating** each field against hardware bounds. If a field is out of range, the last known-good value is substituted (“last-valid hold”) and the appropriate fault bit is set before the correction occurs.

The module maintains private static state (`s_last_speed`, `s_last_temp`, `s_last_gear`, `s_last_mode`) so that the last valid reading is always available for fallback.

4.1.2 Configuration Constants

Table 4.1: Input Module Constants (`input.h`)

Macro	Value	Meaning
<code>INPUT_SPEED_MAX</code>	200	Maximum valid speed in km/h
<code>INPUT_TEMP_MIN</code>	-40	Minimum valid temperature in °C
<code>INPUT_TEMP_MAX</code>	150	Maximum valid temperature in °C
<code>INPUT_GEAR_MAX</code>	5	Maximum valid gear number

4.1.3 Public API

Table 4.2: Input Module Public Functions

Function	Description
<code>read_inputs(VehicleInput *input)</code>	Reads four integers from <code>stdin</code> using <code>scanf</code> . On parse failure, defaults all fields to 0 / <code>MODE_OFF</code> and flushes the input buffer to prevent infinite error loops.
<code>validate_inputs(VehicleInput *input, VehicleStatus *status, FaultStatus *faults)</code>	Validates speed, temperature, gear, and mode in sequence. For gear and mode, the fault bit is set <i>before</i> the value is corrected so that downstream modules see the flag even though the corrected (safe) value is used.

4.1.4 Validation Logic

Safe-hold pattern: On any out-of-range detection, the module retains the last valid value. This ensures the rest of the pipeline always receives a physically plausible input, never garbage.

```

1 if (input->gear > INPUT_GEAR_MAX)
2 {
3     faults->current_cycle_flags |= FAULT_BIT_INVALID_GEAR; /* set BEFORE
4     input->gear = s_last_gear;                               /* replace
5     with safe value */
6 }
7 else
8 {
9     s_last_gear = input->gear; /* update last known-good */
10 }
```

Listing 4.1: Gear validation excerpt from `input.c`

4.2 Mode Module (`mode.h` / `mode.c`)

4.2.1 Responsibility

The mode module implements a deterministic state machine governing the vehicle's operating mode. It enforces *legal transitions only*, blocking any jump that skips an intermediate mode. Illegal requests force the active mode to `MODE_FAULT` and increment the major fault counter.

4.2.2 State Machine

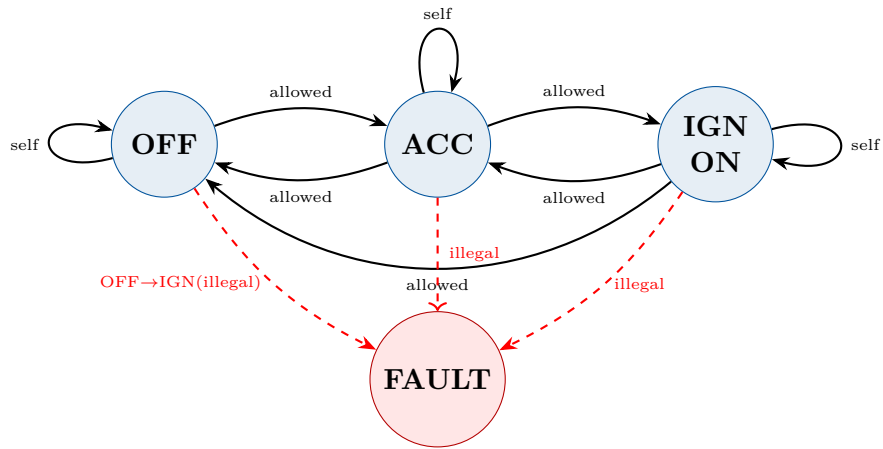


Figure 4.1: Mode state machine — legal transitions and forced FAULT paths

4.2.3 Legal Transition Matrix

Table 4.3: Mode Transition Allowed Matrix (1 = allowed, 0 = illegal → FAULT)

From \ To	OFF	ACC	IGN_ON	FAULT
OFF	1	1	0	—
ACC	1	1	1	—
IGN_ON	1	1	1	—
FAULT	0	0	0	—

4.2.4 Public API

Table 4.4: Mode Module Public Functions

Function	Description
<code>mode_init()</code>	Resets both <code>s_current_mode</code> and <code>s_previous_mode</code> to <code>MODE_OFF</code> . Called by <code>init_system()</code> on startup and reset.
<code>update_mode(status, input, faults)</code>	Validates the requested mode transition. Legal: updates mode history. Illegal or out-of-range: forces <code>MODE_FAULT</code> and increments <code>major_fault_count</code> . Updates <code>status</code> with current and previous modes.
<code>mode_get_current()</code>	Returns the currently active Mode.
<code>mode_get_previous()</code>	Returns the mode active before the last transition.

4.3 Control Module (`control.h` / `control.c`)

4.3.1 Responsibility

The control module is the business-logic layer. It compares the validated sensor inputs against a set of static safety thresholds and sets the appropriate fault bits in `FaultStatus`. It also determines the `highest_priority_issue` field in `VehicleStatus` for diagnostic reporting.

4.3.2 Safety Thresholds

Table 4.5: Control Module Safety Thresholds (`control.h`)

Macro	Value	Effect when exceeded
<code>CONTROL_OVERSPEED_THRESHOLD</code>	120 km/h	Sets <code>FAULT_BIT_OVERSPEED</code> , increments <code>major_fault_count</code>
<code>CONTROL_WARNING_SPEED</code>	110 km/h	Sets <code>FAULT_BIT_WARNING_SPEED</code> , increments <code>major_fault_count</code>
<code>CONTROL_TEMP_CRITICAL_THRESHOLD</code>	110°C	Sets <code>FAULT_BIT_CRITICAL_OVERHEAT</code> , increments <code>major_fault_count</code>
<code>CONTROL_TEMP_HIGH_THRESHOLD</code>	95°C	Sets <code>FAULT_BIT_HIGH_TEMP</code> , increments <code>major_fault_count</code>
<code>CONTROL_WARNING_TEMP</code>	100°C	Sets <code>FAULT_BIT_WARNING_TEMP</code> , increments <code>major_fault_count</code>
<code>CONTROL_TEMP_CRITICAL_THRESHOLD_NEG</code>	-20°C	(Reserved for future cold fault detection)
<code>CONTROL_GEAR_MAX</code>	5	(Reserved for gear range reference)

Temperature evaluation is hierarchical. The conditions use `if / else if` chaining — once `CRITICAL_OVERHEAT` is detected, neither `HIGH_TEMP` nor the warning

path is evaluated for the same cycle.

4.3.3 Fault Priority Logic

After setting fault bits, the control module evaluates the priority of active faults in the following order (highest to lowest):

1. `PRIORITY_CRITICAL_OVERHEAT` — temperature > 110°C
2. `PRIORITY_INVALID_GEAR_MODE` — invalid gear or mode flag set
3. `PRIORITY_OVERSPEED` — speed > 120 km/h
4. `PRIORITY_HIGH_TEMP` — temperature in (95, 110]°C
5. `PRIORITY_NONE` — no active fault

4.3.4 Public API

Table 4.6: Control Module Public Function

Function	Description
<code>run_control_checks(input, status, faults)</code>	Clears the temperature and speed fault bits that it owns, then re-evaluates them from fresh sensor data. Sets fault bits, updates fault counters, and records the highest priority issue.

4.4 Fault Module (`fault.h` / `fault.c`)

4.4.1 Responsibility

The fault module is the persistence tracker. It maintains the `counters[16]` array inside `FaultStatus` — one counter per fault bit position — and decides when a transiently-detected fault becomes a *persistent* (latched) fault.

4.4.2 Persistence Mechanism

A fault is considered persistent when its counter reaches the threshold:

$$\text{counters}[i] \geq \text{FAULT_PERSISTENCE_THRESHOLD} = 3$$

At that point the corresponding bit is OR'd into `persistent_flags`. Persistent flags are *never* cleared automatically; only a full system reset (via `init_system` / `fault_init`) clears them.

If a fault bit is *absent* in the current cycle, its counter is reset to 0, allowing transient faults to clear without latching.

4.4.3 Configuration Constants

Table 4.7: Fault Module Constants (`fault.h`)

Macro	Value	Meaning
<code>FAULT_PERSISTENCE_THRESHOLD</code>	3	Consecutive cycles before a fault is latched
<code>FAULT_MAX_TRACKED_BITS</code>	5	Number of primary fault bits iterated in <code>update_fault_s</code>

4.4.4 Public API

Table 4.8: Fault Module Public Functions

Function	Description
<code>fault_init(faults)</code>	Zeroes all fields: <code>current_cycle_flags</code> , <code>persistent_flags</code> , and all 16 counters.
<code>set_fault(faults, fault_bit)</code>	OR's <code>fault_bit</code> into <code>current_cycle_flags</code> .
<code>clear_fault(faults, fault_bit)</code>	AND-NOT clears <code>fault_bit</code> from <code>current_cycle_flags</code> .
<code>increment_fault_counter(faults, index)</code>	Safely increments <code>counters[index]</code> .
<code>update_fault_status(faults)</code>	Main persistence engine. Iterates bits 0 through <code>FAULT_MAX_TRACKED_BITS-1</code> ; if active, increments counter and latches to <code>persistent_flags</code> when threshold is met; if inactive, resets counter to 0.

```

1 for (uint8_t i = 0U; i < FAULT_MAX_TRACKED_BITS; i++)
2 {
3     uint16_t current_mask = (uint16_t)(1U << i);
4
5     if ((faults->current_cycle_flags & current_mask) != 0U)
6     {
7         increment_fault_counter(faults, i);
8
9         if (faults->counters[i] >= FAULT_PERSISTENCE_THRESHOLD)
10        {
11            faults->persistent_flags |= current_mask; /* latch */
12        }
13    }
14    else
15    {

```

```

16     faults->counters[i] = 0U;  /* clear transient counter */
17 }
18 }

```

Listing 4.2: Core persistence loop from fault.c

4.5 State Module (state.h / state.c)

4.5.1 Responsibility

The state module drives the top-level system health state machine. It evaluates the fault counts accumulated during a cycle and escalates or recovers the `SystemState` accordingly.

4.5.2 State Machine

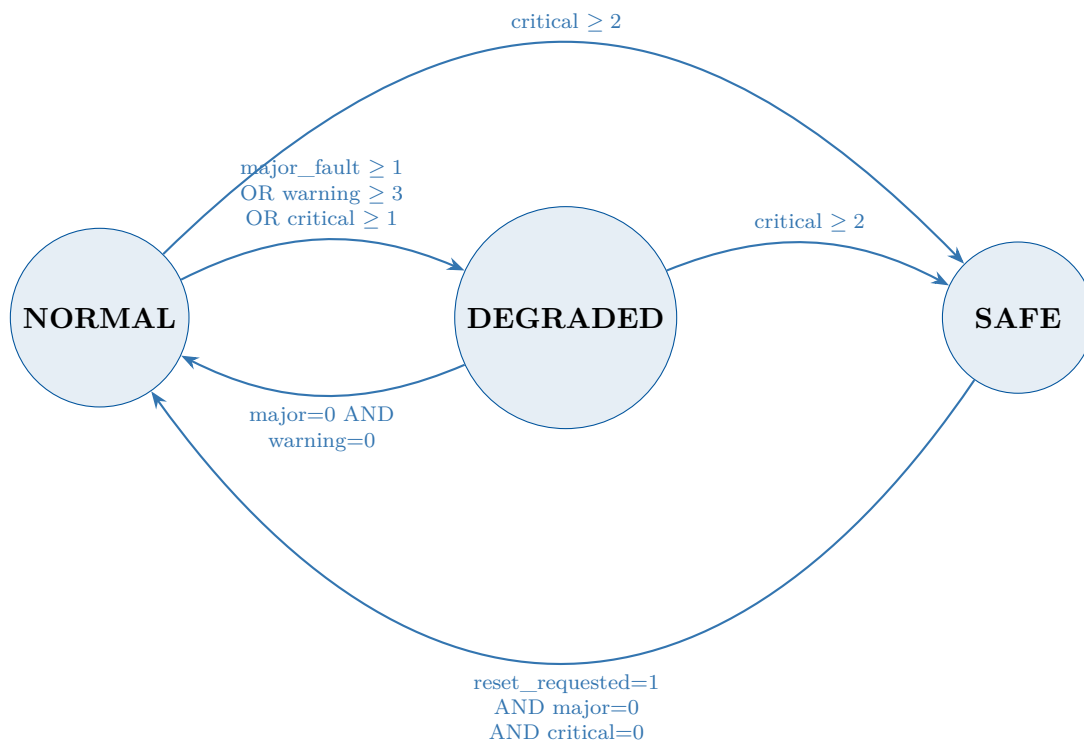


Figure 4.2: System state machine — transition conditions

4.5.3 State Transition Rules

Table 4.9: State Transition Conditions

From	To	Condition
NORMAL	SAFE	<code>critical_fault_count</code> $\geq$ 2
NORMAL	DEGRADED	<code>major_fault_count</code> $\geq$ 1 OR <code>warning_count</code> $\geq$ 3 OR <code>critical_fault_count</code> $\geq$ 1
DEGRADED	SAFE	<code>critical_fault_count</code> $\geq$ 2
DEGRADED	NORMAL	<code>major_fault_count</code> = 0 AND <code>warning_count</code> = 0
SAFE	NORMAL	<code>reset_requested</code> = 1 AND <code>major_fault_count</code> = 0 AND <code>critical_fault_count</code> = 0

4.5.4 Configuration Constants

Table 4.10: State Module Thresholds (`state.h`)

Macro	Value	Meaning
<code>STATE_MAJOR_FAULT_THRESHOLD</code>	1	1 major fault triggers DEGRADED
<code>STATE_WARNING_REPEAT_THRESHOLD</code>	3	3 warnings triggers DEGRADED
<code>STATE_CRITICAL_FAULT_THRESHOLD</code>	2	2 critical faults trigger SAFE

4.5.5 Public API

Table 4.11: State Module Public Functions

Function	Description
<code>init_system(status, faults)</code>	Master initialiser. Resets the internal state to NORMAL , resets status and faults to safe defaults, and calls <code>mode_init()</code> and <code>fault_init()</code> .
<code>evaluate_system_state(status, faults)</code>	Runs the state machine switch-case. Applies transitions based on fault counts and updates both the internal static state and <code>status->system_state</code> .
<code>state_get_current()</code>	Returns the internal SystemState without modifying it.
<code>state_request_reset(faults)</code>	Clears all fault count fields and sets <code>reset_requested = 1</code> , arming the state machine to exit SAFE on the next evaluation.

4.6 Log Module (`log.h` / `log.c`)

4.6.1 Responsibility

The log module is the final stage of every pipeline cycle. It formats and prints a comprehensive diagnostic summary to `stdout` and simultaneously appends structured records to two persistent log files:

- `valid_input.txt` — timestamped record of every validated sensor snapshot.
- `faults_errors.txt` — timestamped record of every cycle’s fault bitmask and counters.

4.6.2 Cycle Summary Sections

Each call to `log_cycle_summary()` produces the following sections:

1. **[INPUT]** — Speed (km/h), Temperature (°C), Gear, Requested Mode.
2. **[MODE]** — Active Mode, Current Mode, Previous Mode.
3. **[FAULTS]** — Cycle flags bitmask (5-bit display), Persistent flags bitmask, Major/Warning/Critical counts, Reset flag.

4. **[STATE]** — Current System State (**NORMAL** / **DEGRADED** / **SAFE**).
5. **[ACTIVE FAULTS — Priority Order]** — Lists all active faults ranked by priority.

4.6.3 Fault Flag Display Format

The 5-bit display in the fault section encodes bits in the following order (MSB to LSB of the display string):

$$\left[\underbrace{B_4}_{\text{INVALID_MODE}} \underbrace{B_3}_{\text{INVALID_GEAR}} \underbrace{B_2}_{\text{HIGH_TEMP}} \underbrace{B_1}_{\text{CRIT_OVERHEAT}} \underbrace{B_0}_{\text{OVERSPEED}} \right]$$

4.6.4 Public API

Table 4.12: Log Module Public Function

Function	Description
<code>log_cycle_summary(input, status, faults)</code>	Prints the full cycle summary to <code>stdout</code> . Appends input record to <code>valid_input.txt</code> and fault record to <code>faults_errors.txt</code> using <code>fopen("filename", "a")</code> . Timestamp is obtained via <code>time()</code> and formatted with <code>strftime()</code> .

Chapter 5

Main Execution Pipeline (`main.c`)

5.1 Entry Point

`src/main.c` contains the sole `main()` function of the simulator. It owns the top-level data structures and drives the cyclic pipeline. All module calls are made by passing pointers to `input`, `status`, and `faults` — no global variables are shared.

5.2 Watchdog Logic

The simulator implements a dual-condition software watchdog:

1. **All-zero sentinel:** If the operator inputs 0 0 0 0, the main loop triggers `init_system()` immediately on that cycle, resets the safe-mode counter, and forces the input to safe defaults so the reset cycle evaluates cleanly as `NORMAL`.
2. **Consecutive SAFE cycles:** The variable `safe_mode_cycle_count` increments each time the system state evaluates to `SAFE`. When it reaches 2, a watchdog reset is triggered automatically — simulating automatic limp-home initiation.

```
1 if ((input.speed == 0 && input.temperature == 0 &&
2     input.gear == 0 && input.mode == 0) ||
3     (safe_mode_cycle_count >= 2))
4 {
5     /* ... trigger init_system() and reset ... */
6 }
```

Listing 5.1: Watchdog trigger condition from `main.c`

5.3 Local Variables

Table 5.1: `main.c` Local Variables

Variable	Type	Purpose
<code>input</code>	<code>VehicleInput</code>	Sensor snapshot for each cycle
<code>status</code>	<code>VehicleStatus</code>	Mode and system state
<code>faults</code>	<code>FaultStatus</code>	Fault tracking state
<code>safe_mode_cycle_count</code>	<code>uint8_t</code>	Consecutive SAFE-state counter for watchdog

Chapter 6

Build and Usage

6.1 Requirements

- GCC compiler (version 5.0 or later recommended)
- Linux, macOS, or Windows Subsystem for Linux (WSL)

6.2 Compilation

The project uses strict compiler flags to enforce MISRA-relevant warnings:

```
1 gcc -Wall -Wextra -pedantic -std=c99 -Iinclude \  
2 -o ecu_sim \  
3 src/main.c src/input.c src/mode.c \  
4 src/control.c src/fault.c src/state.c src/log.c
```

Listing 6.1: Compilation command

Table 6.1: Compiler Flags

Flag	Purpose
-Wall	Enable standard warnings
-Wextra	Enable additional warnings
-pedantic	Enforce strict ISO C99 conformance
-std=c99	Use ISO C99 standard
-Iinclude	Add the include/ directory to the header search path

6.3 Running the Simulator

```
1 ./ecu_sim
```

Listing 6.2: Running the simulator

The simulator will start, print the banner, initialise the system, and prompt for input each cycle:

```
=====
VEHICLE ECU SIMULATOR
=====
[MAIN] System initialisation complete.

--- NEW CYCLE ---

[Input] Enter Speed, Temp, Gear, Mode (0=OFF, 1=ACC, 2=IGN_ON, 3=FAULT):
```

6.4 Input Format

Format: [Speed] [Temp] [Gear] [Mode]

Table 6.2: Input Field Specification

Field	Type	Valid Range	Mode Mapping
Speed	integer	0 – 200 km/h	—
Temperature	integer	-40 – 150°C	—
Gear	integer	0 – 5	—
Mode	integer	0 – 3	0=OFF, 1=ACC, 2=IGN_ON, 3=FAULT

6.5 Walkthrough: Normal Vehicle Start-Up

Important: Modes must be escalated one step at a time. Jumping from **OFF** directly to **IGN_ON** is an illegal transition and will force **MODE_FAULT**.

Table 6.3: Example: Normal Start-Up Sequence

Cycle	Input	Active Mode	System State
1	0 80 0 1	ACC	NORMAL
2	40 80 1 2	IGNITION_ON	NORMAL

6.6 Walkthrough: Fault Scenarios

Table 6.4: Key Fault Scenarios

Cycle	Input	Fault Detected	Resulting State
3	130 85 4 2	OVERSPEED	DEGRADED (major fault)
4	40 100 4 2	HIGH_TEMP	DEGRADED (major fault)
5	40 115 4 2	CRITICAL_OVERHEAT	SAFE (critical fault)
6	40 80 9 2	INVALID_GEAR	Input corrected to last valid
—	0 0 0 2 (from OFF)	ILLEGAL_MODE	Active mode forced to FAULT

6.7 Watchdog Reset Procedure

To trigger a graceful system reset:

1. Send the all-zero sentinel: 0 0 0 0
2. OR, allow the system to remain in **SAFE** state for two consecutive cycles — the watchdog will auto-reset.

After a reset, the system returns to **NORMAL** with mode **OFF** and all fault counters cleared.

Chapter 7

Safety and Coding Standards

7.1 MISRA-C Compliance Practices

The following MISRA-C practices have been applied throughout the codebase:

- **No magic numbers:** All numeric limits are defined as named macros (e.g., `FAULT_PERSISTENCE_THRESHOLD`, `CONTROL_OVERSPEED_THRESHOLD`).
- **Explicit integer types:** `<stdint.h>` types (`uint8_t`, `uint16_t`, `int16_t`) are used throughout to prevent implicit type conversion errors.
- **Safe bitwise operations:** All bitwise clears use the AND-NOT pattern: `flags &= ~mask;` (Rule 10.1).
- **Unsigned literals:** Shift operations and bitmask comparisons use the U suffix (e.g., `1U << i`).
- **Explicit parentheses:** Complex boolean and bitwise expressions are fully parenthesised.

7.2 CERT-C Compliance Practices

- **MEM10-C — NULL Pointer Validation:** Every public function validates all pointer parameters before dereferencing them and returns immediately (with an error to `stderr`) if any pointer is `NULL`.
- **Bounds Checking:** Array access in `fault.c` is bounded by the `FAULT_MAX_TRACKED_BITS` constant, preventing out-of-bounds reads.
- **Input Sanitisation:** `scanf()` return value is checked; on failure, the input buffer is flushed to prevent an infinite re-read loop.

7.3 Defensive Design Patterns

Table 7.1: Defensive Programming Patterns Used

Pattern	Where Applied
Last-valid hold	<code>input.c</code> : retains previous sensor reading when current reading is out of range
Fault-before-correct	<code>input.c</code> : gear and mode fault bits are set <i>before</i> the field is corrected so downstream modules see the fault flag
State machine guards	<code>mode.c</code> and <code>state.c</code> : all unknown states fall through to a safe default (FAULT or SAFE)
Dual-path watchdog	<code>main.c</code> : system reset triggered by either explicit sentinel or automatic SAFE-state detection
Persistent latching	<code>fault.c</code> : faults can only be cleared by an explicit system reset, not by a single fault-free cycle

Chapter 8

Output Log Files

8.1 valid_input.txt

Every cycle's validated sensor snapshot is appended to this file with a timestamp. The format is:

```
[INPUT] - YYYY-MM-DD HH:MM:SS
  Speed      : <value> km/h
  Temperature : <value> C
  Gear       : <value>
  Requested Mode : <mode string>
```

8.2 faults_errors.txt

Every cycle's fault state is appended to this file:

```
[FAULTS] - YYYY-MM-DD HH:MM:SS
  Cycle Flags : <5-bit binary> [ <fault names> ]
  Persistent  : <5-bit binary> [ <fault names> ]
  Major Faults : <value>
  Warnings     : <value>
  Critical Faults : <value>
  Reset Requested : YES / NO
```

Chapter 9

Glossary

Term	Definition
ECU	Electronic Control Unit — the embedded computer managing vehicle functions.
Cyclic Pipeline	An execution pattern where a fixed sequence of steps repeats indefinitely (each iteration = one “cycle”).
MISRA-C	Motor Industry Software Reliability Association — a set of C language usage guidelines for safety-critical software.
CERT-C	Carnegie Mellon University’s secure coding standard for C.
Fault Bit	A single bit in a bitmask that records the presence of a specific fault condition.
Persistent Fault	A fault that has occurred in three consecutive cycles and has been latched; requires a full reset to clear.
Watchdog	A circuit or software mechanism that triggers a system reset when a fault or unsafe condition persists.
Last-valid hold	A sensor safety technique where the last in-range reading is retained when the current reading is out of range.
NORMAL	System state indicating no active faults.
DEGRADED	System state indicating reduced operation due to one or more faults.
SAFE	Terminal system state requiring a reset before normal operation can resume.
MODE_FAULT	Active mode value set when an illegal mode transition is attempted.
Bitwise Latch	OR-ing a fault bit into a persistent register so it cannot be cleared by a single clean cycle.

Table 9.1: Glossary of Terms